1. Paper Title

A Study on Image Processing Using Mathematical Morphological

2. Method / Technique Applied

The technique applied in this work is Binary Mathematical Morphology, based on the methodology described in the reference paper.
The implemented workflow is: **Input Image → Input Preparation / Binarization → Binary Image → Rectangular Structuring Element → Erosion → Dilation → Opening → Closing → White Top-Hat → Black Top-Hat → Final Mathematical Morphological Output**

The main mathematical morphological operations implemented are:

1. **Erosion**
2. **Dilation**
3. **Opening**
4. **Closing**
5. **White Top-Hat (WTH)**
6. **Black Top-Hat (BTH)**
7. **Final morphological recombination**

The primary processing mode of the implementation is binary morphology, matching the binary-image workflow of the reference paper.

Mathematical Definitions

1. **Erosion**: Erosion removes foreground pixels according to the shape and size of the structuring element.
2. **Dilation**: Dilation expands the foreground region according to the structuring element.
3. **Opening**: Opening is defined as erosion followed by dilation: **Opening(I, B) = (I $\ominus$ B) $\oplus$ B**
    1. where:
    2. I = input binary image
    3. B = structuring element
    4. $\ominus$ = erosion
    5. $\oplus$ = dilation
4. **Closing:** Closing is defined as dilation followed by erosion: **Closing(I, B) = (I $\oplus$ B) $\ominus$ B**
5. **White Top-Hat: WTH = I − Opening(I, B)**
6. **Black Top-Hat: BTH = Closing(I, B) − I**
7. **Final Mathematical Morphological Output: Final Output = I + WTH − BTH**

The implementation uses a rectangular structuring element. The primary paper-oriented configuration is a **4 × 8** rectangular structuring element.
The reference paper also evaluates **3 × 8, 4 × 8, 5 × 8, and 6 × 8** rectangular structuring elements.

Custom Implementation Requirement

The main image-processing operations were implemented using custom Python functions rather than existing built-in morphology functions.

The implementation does not use:

- **cv2.erode()**
- **cv2.dilate()**
- **cv2.morphologyEx()**
- **scipy.ndimage.binary_erosion()**
- **scipy.ndimage.binary_dilation()**
- **skimage.morphology operations**

Instead, the implementation explicitly processes the active positions of the structuring element and performs the required logical operations using NumPy array operations.

Custom functions include:

- **erosion_binary()**
- **dilation_binary()**
- **opening_binary()**
- **closing_binary()**
- **white_top_hat_binary()**
- **black_top_hat_binary()**
- **recombine_binary()**

Custom input-preparation functions are also used for grayscale conversion and Otsu threshold calculation.

3. Algorithm / Pseudocode

Input

- RGB or grayscale image
- Rectangular structuring element

Procedure

1. Load the input image dynamically.
2. Validate the image format and dimensions.
3. Convert RGB input to grayscale using the custom grayscale conversion function when necessary.
4. Calculate the threshold using the custom Otsu thresholding implementation.
5. Convert the grayscale image into a binary image.
6. Create the rectangular structuring element.
7. Set the primary structuring element to 4 × 8 for the paper-oriented configuration.
8. Perform custom binary erosion.
9. Perform custom binary dilation.
10. Calculate opening using erosion followed by dilation.
11. Calculate closing using dilation followed by erosion.
12. Calculate White Top-Hat using the input image and opening result.
13. Calculate Black Top-Hat using the closing result and input image.
14. Recombine the results using: **Final Output = Input + WTH − BTH**
15. Compare the different rectangular structuring-element configurations.
16. Perform consistency and sanity checks.
17. Display the intermediate and final results.
18. Calculate quantitative foreground-pixel statistics.

4. Implementation Procedure

4.1 Dynamic Input Image

The implementation supports dynamic image input rather than depending on a fixed hardcoded filename.
The notebook supports image upload through Google Colab and also provides a local-path fallback.

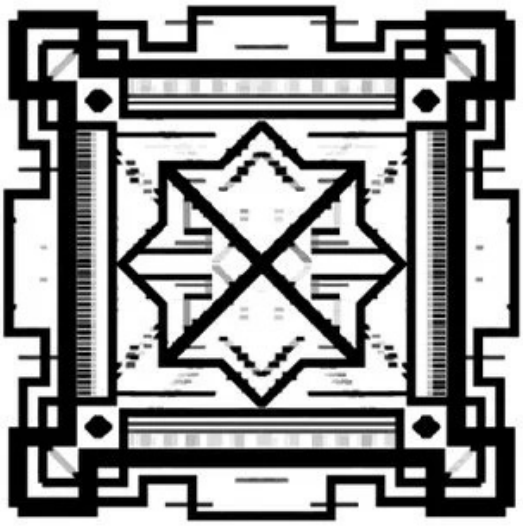


Figure 1: Original Input Image

Supported image formats include: **JPG, JPEG, PNG, BMP, TIFF, TIF, WEBP**
This makes the notebook reusable with different input images.

4.2 RGB-to-Grayscale Conversion

When an RGB image is provided, the notebook converts it to grayscale using a custom implementation.
The grayscale intensity is calculated using: **Gray = 0.299R + 0.587G + 0.114B**
This operation is implemented through a custom function rather than relying on a built-in image-processing grayscale conversion operation.

4.3 Custom Otsu Thresholding

Because the primary mathematical morphology workflow operates on a binary image, the input image is converted into binary form before morphology is applied.
The notebook implements Otsu threshold selection using a custom function: **otsu_threshold(gray)**
The function calculates the grayscale histogram, evaluates possible thresholds, computes the between-class variance, and selects the threshold that maximizes the separation between foreground and background classes.
For the current executed input:

- Image size: **320 × 320 pixels**
- Otsu threshold: **130**
- Foreground pixels: **63,383**
- Background pixels: **39,017**

The Otsu operation is considered input preparation, because the reference paper's morphological workflow starts from a binary image.

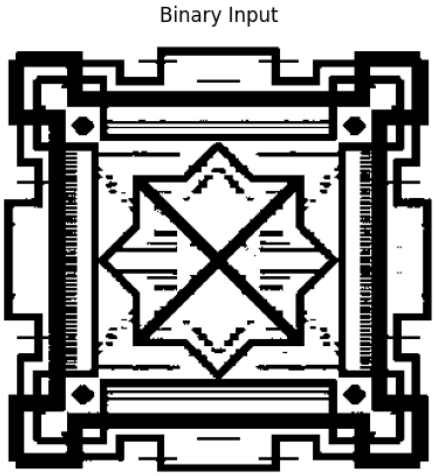


Figure 2: Binary Image Obtained After Custom Otsu Thresholding

5. Custom Mathematical Morphology

5.1 Binary Erosion

Binary erosion is implemented using a custom function.
The function processes the active positions of the structuring element and evaluates the corresponding image locations.
A foreground pixel remains foreground only when the required structuring-element positions satisfy the erosion condition.
Conceptually: **Erosion(I, B) = {x | Bx ⊆ I}**
The implementation performs the operation through explicit spatial shifting and logical **AND** operations.

5.2 Binary Dilation

Binary dilation is implemented using a custom function.
The active positions of the structuring element are shifted across the image, and the corresponding regions are combined using logical OR operations.
Conceptually: **Dilation(I, B) = {x | Bx ∩ I ≠ ∅}**
This allows the dilation operation to be implemented without using a built-in morphology function.

5.3 Opening

Opening is implemented using the custom erosion and dilation functions.
The operation is: **Opening(I, B) = Dilation(Erosion(I, B), B)**
Opening generally removes small foreground structures and smooths the boundaries of larger foreground objects.

5.4 Closing

Closing is implemented using the custom dilation and erosion functions.
The operation is: **Closing(I, B) = Erosion(Dilation(I, B), B)**
Closing generally fills small gaps and holes and connects nearby foreground regions depending on the structuring-element geometry.

5.5 White Top-Hat

White Top-Hat is calculated using: **WTH = I − Opening(I, B)**
It represents foreground details that are removed by the opening operation.
The implementation calculates this result from the original binary image and the custom opening output.

5.6 Black Top-Hat

Black Top-Hat is calculated using: **BTH = Closing(I, B) − I**
It represents background details or dark regions that are filled or modified by the closing operation.
The implementation calculates this result from the custom closing output and the original binary image.

5.7 Final Recombination

The final mathematical morphological output is calculated as: **Final Output = I + WTH − BTH**
The implementation performs this recombination using the custom results produced by the preceding morphological operations.
For the binary configuration used in the current execution, the recombination reconstructs the original binary image exactly, which is confirmed by the sanity check.

6. Current Experimental Configuration

The current notebook execution uses the following primary configuration:

Parameter	**Current Configuration**
Processing Mode	Binary
Structuring Element Mode	Paper
Structuring Element	4 × 8
Structuring Element Type	Rectangular
Thresholding	Custom Otsu
Image Size	320 × 320
Otsu Threshold	130
Input Foreground Pixels	63,383

The 4 × 8 rectangular structuring element is retained as the primary paper-oriented configuration.
The notebook also supports comparison of the other rectangular configurations evaluated in the reference paper.

7. Output Images

The notebook produces a complete visualization of the morphological processing pipeline.
The output visualization contains:

1. Original Grayscale Image
2. Binary Input Image
3. Erosion
4. Dilation
5. Opening
6. Closing
7. White Top-Hat
8. Black Top-Hat
9. Final Mathematical Morphological Output

These intermediate outputs make it possible to visually observe the effect of each mathematical morphology operation.

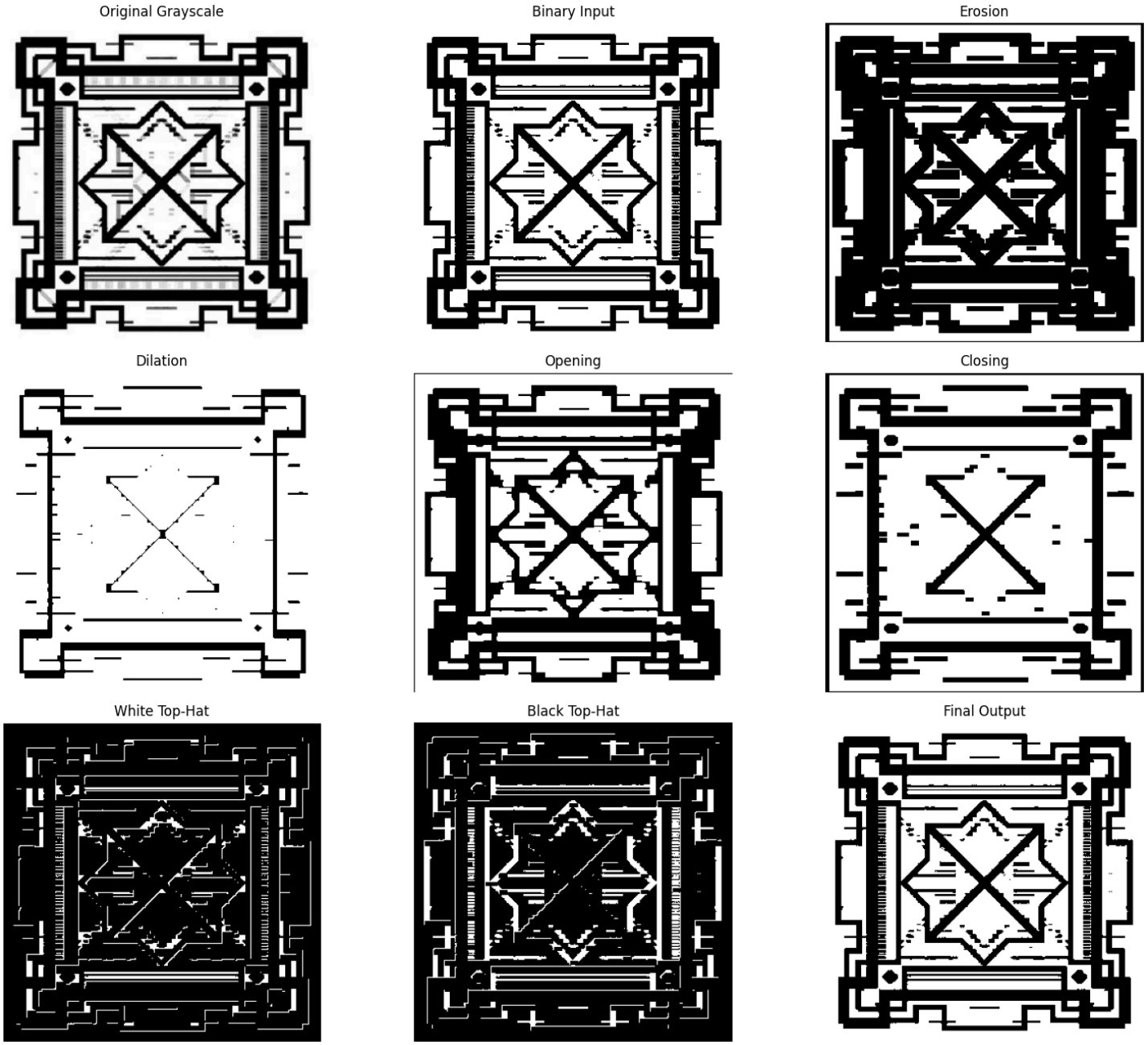


Figure 3: Intermediate and Final Results of Mathematical Morphological Processing

8. Quantitative Results

The following results were obtained from the current execution using the 4 × 8 rectangular structuring element.

Operation	Foreground Pixel Count
Input Binary Image	63,383
Erosion	32,754
Dilation	90,256
Opening	57,062
Closing	72,513
White Top-Hat	10,555
Black Top-Hat	15,166
Final Output	63,383

The erosion result contains fewer foreground pixels because erosion removes boundary regions according to the structuring element.
The dilation result contains more foreground pixels because dilation expands foreground regions.
Opening produces 57,062 foreground pixels, while closing produces 72,513 foreground pixels.
The White Top-Hat result contains 10,555 foreground pixels, representing foreground information removed during opening.
The Black Top-Hat result contains 15,166 foreground pixels, representing information introduced by the closing operation relative to the input.
The final output contains 63,383 foreground pixels, equal to the original binary input.

9. Structuring Element Comparison

The notebook also evaluates the rectangular structuring-element sizes used in the reference paper.

Structuring Element	Opening	Closing	WTH	BTH
3 × 8	58,221	71,058	6,887	11,594
4 × 8	57,062	72,513	10,555	15,166
5 × 8	55,594	73,187	9,421	14,130
6 × 8	54,622	77,943	12,443	20,140

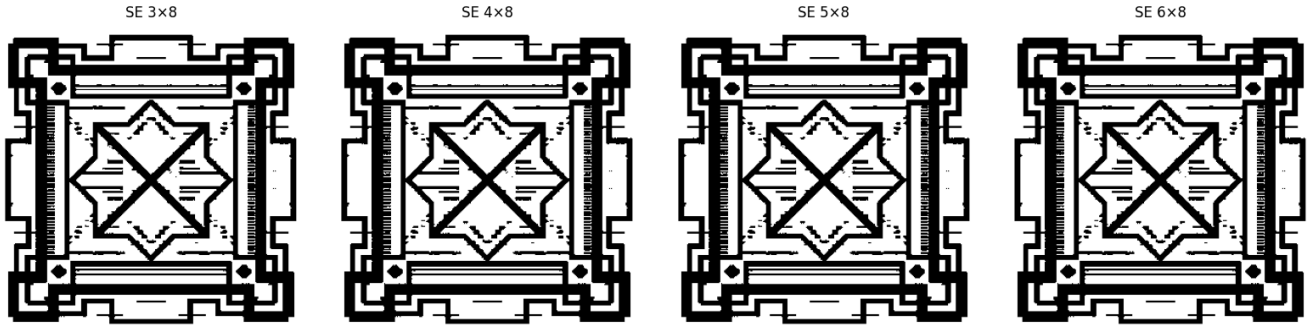


Figure 4: Comparison of Morphological Results Using Different Rectangular Structuring Elements

The **4 × 8** configuration is retained as the primary paper-oriented configuration.
The other configurations are included to reproduce the comparison of rectangular structuring elements and to demonstrate how structuring-element dimensions affect the morphological output.
No universal claim is made that 4 × 8 is the best structuring element for every possible image. Its selection here follows the reference-paper configuration for the particular image-processing experiment.

10. Verification and Sanity Checks

The notebook includes verification mechanisms to ensure that the custom implementations behave consistently.

10.1 Binary Recombination Check

The final recombined output is compared against the original binary input.
Results:

- Mismatch pixels: **0**
- Mismatch rate: **0%**
- Maximum absolute difference: **0**

This confirms that the final binary recombination produces the same binary image for the current execution.

10.2 Opening Consistency Check

The opening result is checked against the equivalent sequence:
Opening = Erosion → Dilation
Results:

- Mismatch pixels: **0**
- Mismatch rate: **0%**
- Maximum difference: **0**

10.3 Closing Consistency Check

The closing result is checked against the equivalent sequence:
Closing = Dilation → Erosion
Results:

- Mismatch pixels: **0**
- Mismatch rate: **0%**
- Maximum difference: **0**

These checks provide evidence that the custom morphology functions are internally consistent.

11. Additional Features and Engineering Improvements

In addition to the primary requirements, the notebook contains several features that improve its usability and reusability.

11.1 Dynamic Image Upload

The notebook allows the user to upload an image dynamically rather than modifying a hardcoded filename.
This makes the implementation suitable for testing different input images.

11.2 Custom RGB-to-Grayscale Conversion

The notebook includes its own RGB-to-grayscale conversion implementation.
This ensures that the preprocessing stage is also explicitly controlled rather than relying entirely on a library-provided image-processing conversion.

11.3 Custom Otsu Thresholding

Otsu thresholding is implemented manually.
The implementation calculates the histogram and evaluates the threshold using between-class variance.
This provides a reproducible and transparent binarization process.

11.4 Adaptive Structuring Element

The notebook includes an adaptive structuring-element mode.
The adaptive mode selects a rectangular structuring element according to the input image dimensions while maintaining an approximately 1:2 height-to-width relationship and applying dimension bounds.
This feature improves the reusability of the implementation for images with different resolutions.
The adaptive structuring element is an engineering extension and is not presented as a method proposed by the reference paper.

11.5 Multiple Structuring-Element Experiments

The notebook supports the comparison of:

- **3 × 8**
- **4 × 8**
- **5 × 8**
- **6 × 8**

This allows the effect of different structuring-element sizes to be observed quantitatively.

11.6 Optional Grayscale Morphology

The notebook also contains an optional grayscale morphology implementation.
This is separate from the primary binary morphology workflow.
The grayscale implementation uses intensity-based minimum and maximum operations to perform grayscale erosion and dilation, followed by grayscale opening, closing, White Top-Hat, and Black Top-Hat operations.

The grayscale morphology implementation is provided as an additional engineering extension and is not presented as the primary reproduction of the reference paper.

11.7 Configurable Processing Modes

The notebook provides configurable modes such as:

PROCESSING_MODE = "**binary**"
SE_MODE = "**paper**"
PAPER_SE = (**4, 8**)

The primary configuration is:

PROCESSING_MODE = "**binary**"
SE_MODE = "**paper**"
PAPER_SE = (**4, 8**)

Optional configurations include:

PROCESSING_MODE = "**grayscale**"
and:
SE_MODE = "**adaptive**"

This lets you reuse the same notebook for different experimental configurations without rewriting the core implementation.

11.8 Display and Visualization

The notebook provides visual output for each major processing stage.
Display-only scaling is used where necessary to make binary or morphological images visible in plots.
This visualization scaling does not modify the underlying processing data.

12. Results and Analysis

The experimental results demonstrate the expected behavior of the implemented mathematical morphology operations.
The input binary image contains 63,383 foreground pixels.
After erosion, the foreground count decreases to 32,754 pixels. This reduction occurs because erosion removes pixels around foreground boundaries according to the shape of the 4 × 8 structuring element.
After dilation, the foreground count increases to 90,256 pixels, demonstrating the expansion of foreground regions.
Opening produces 57,062 foreground pixels. Because opening is erosion followed by dilation, it removes small foreground structures and boundary details.
Closing produces 72,513 foreground pixels. Closing can fill small gaps and connect nearby foreground regions depending on the structuring element.
The White Top-Hat result contains 10,555 pixels, while the Black Top-Hat result contains 15,166 pixels.
The final recombination produces 63,383 foreground pixels, matching the original binary input.
The zero-mismatch verification confirms that the final binary recombination is internally consistent for the current input.

13. Discussion

The primary contribution of the implementation is the custom realization of the binary mathematical morphology workflow.

Instead of directly calling built-in image-processing morphology functions, the implementation provides explicit control over:

- Structuring-element positions
- Spatial shifting
- Boundary handling
- Binary erosion
- Binary dilation
- Opening
- Closing
- White Top-Hat
- Black Top-Hat
- Final recombination

This approach directly addresses the assignment requirement that the main image-processing operations be implemented using custom functions.

The additional features improve the reusability and experimental capability of the notebook.

Feature	**Classification**
Binary Mathematical Morphology	Primary Assignment Method
4 × 8 Rectangular SE	Primary Paper Configuration
3 × 8 / 4 × 8 / 5 × 8 / 6 × 8	Reference-Based Comparison
Custom Otsu Thresholding	Input Preparation
Custom RGB-to-Grayscale	Input Preparation
Dynamic Image Upload	Reusability Feature
Adaptive Structuring Element	Engineering Extension
Grayscale Morphology	Engineering Extension
Quantitative Measurements	Analysis Feature
Sanity and Consistency Checks	Verification Feature

The distinction between the primary method and the additional features is important. The binary morphology workflow is the main implementation corresponding to the reference paper, while adaptive structuring elements and grayscale morphology are additional engineering capabilities of the notebook.

14. Conclusion

This work implements a complete binary mathematical morphology pipeline using custom Python functions and a rectangular structuring element.
The primary workflow consists of: **Binary Input → Erosion → Dilation → Opening → Closing → White Top-Hat → Black Top-Hat → Final Recombination**
The primary experimental configuration uses a **4 × 8** rectangular structuring element, while 3 × 8, 5 × 8, and 6 × 8 configurations are also evaluated for comparison.
The main morphological operations are implemented through custom functions rather than built-in morphology functions. The implementation therefore provides direct control over the mathematical operations and satisfies the requirement to create the required image-processing operations independently.

The notebook also includes several additional engineering features, including:

- Dynamic image upload
- Multiple image-format support
- Custom RGB-to-grayscale conversion
- Custom Otsu thresholding
- Configurable processing modes
- Adaptive structuring-element selection
- Multiple structuring-element experiments
- Optional grayscale morphology
- Intermediate visualization
- Quantitative analysis
- Automated consistency checks

For the current 320 × 320 input image, the custom implementation produced a binary input with 63,383 foreground pixels and a final recombined output with the same number of foreground pixels.
The verification results produced zero mismatched pixels for the final recombination, opening consistency, and closing consistency checks.
The additional adaptive and grayscale features are presented as engineering extensions and are not attributed to the reference paper. The primary experimental workflow remains the binary mathematical morphology method based on the reference paper.

15. Reference Paper

Authors: **Khairul Anuar Mat Said and Asral Bahari Jambek**
Title: **A Study on Image Processing Using Mathematical Morphological**
Conference: 2016 3rd International Conference on Electronic Design (ICED)
Date: 11–12 August 2016
Location: Phuket, Thailand
Affiliation: School of Microelectronic Engineering, Universiti Malaysia Perlis, Malaysia

The paper presents mathematical morphology using rectangular structuring elements, including **erosion, dilation, opening, closing, White Top-Hat, and Black Top-Hat**.

The implementation follows the paper's binary mathematical morphology workflow while adding **custom input handling, adaptive SE selection, optional grayscale morphology, quantitative analysis, and automated verification**.